

FINAL PROJECT REPORT

SEMESTER 2, ACADEMIC YEAR: 2024-2025

CT312H: MOBILE PROGRAMMING

- **Project/Application name:** Nutrition Tracker Recipes
- **GitHub link:** <https://github.com/24-25Sem2-Courses/ct312hm04-project-LamSut>
- **Youtube link:** https://youtu.be/R_yRUsnouYg
- **Student ID 1:** B2111933
- **Student Name 1:** Truong Dang Truc Lam
- **Student ID 2:** B2111984
- **Student Name 2:** Dang Hoang Hung
- **Class/Group Number:** CT312HM04

I. Introduction

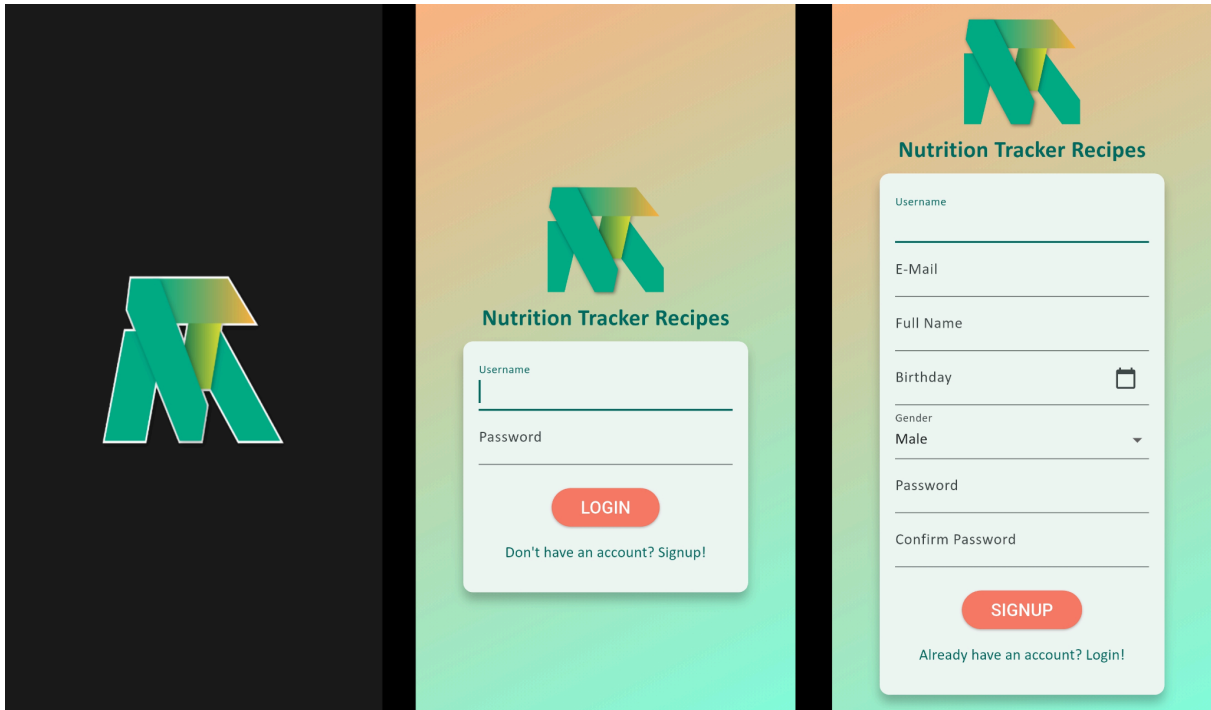
- Project/application description: A Flutter mobile application for exploring foods with nutrient data and creating personalized recipes, using PocketBase for backend storage.
- A task assignment sheet for each member if working in groups.

No.	Task Description	Member(s)
1	Initialize the project with shared utils	Truong Dang Truc Lam
2	Design the authentication pages	Truong Dang Truc Lam Dang Hoang Hung
3	Design the foods pages	Truong Dang Truc Lam Dang Hoang Hung
4	Design the recipes pages	Truong Dang Truc Lam
5	Design the user information pages	Dang Hoang Hung
6	Setup Pocketbase for data storage	Truong Dang Truc Lam
7	Design the app icon & splash screen	Dang Hoang Hung

II. Details of implemented features

1. Authentication

- **Description:** This feature allows users to sign up and log in to access further features and store their information under their account.
- **Screenshots:**



Splash, Login and Sign Up screens

- **Implementation details:**
 - Widgets used for this feature:
 - **UserAuthScreen** (Main screen)
 - Scaffold, Stack, Container (Gradient background)
 - SingleChildScrollView, SizedBox, Column (Layout structure)
 - Image.asset (App icon), Text (App title)
 - Flexible (Responsive AuthCard)
 - **AuthCard** (Authentication form)
 - Card, Container (Styled form container)
 - Form, SingleChildScrollView, Column (Form layout)
 - TextFormField (Username, Email, Password,...)
 - DropdownButtonFormField (Gender selection)
 - IconButton (Date picker for birthday)
 - ElevatedButton (Submit button)
 - TextButton (Switch login/signup mode)
 - ValueListenableBuilder, CircularProgressIndicator (Loading state)

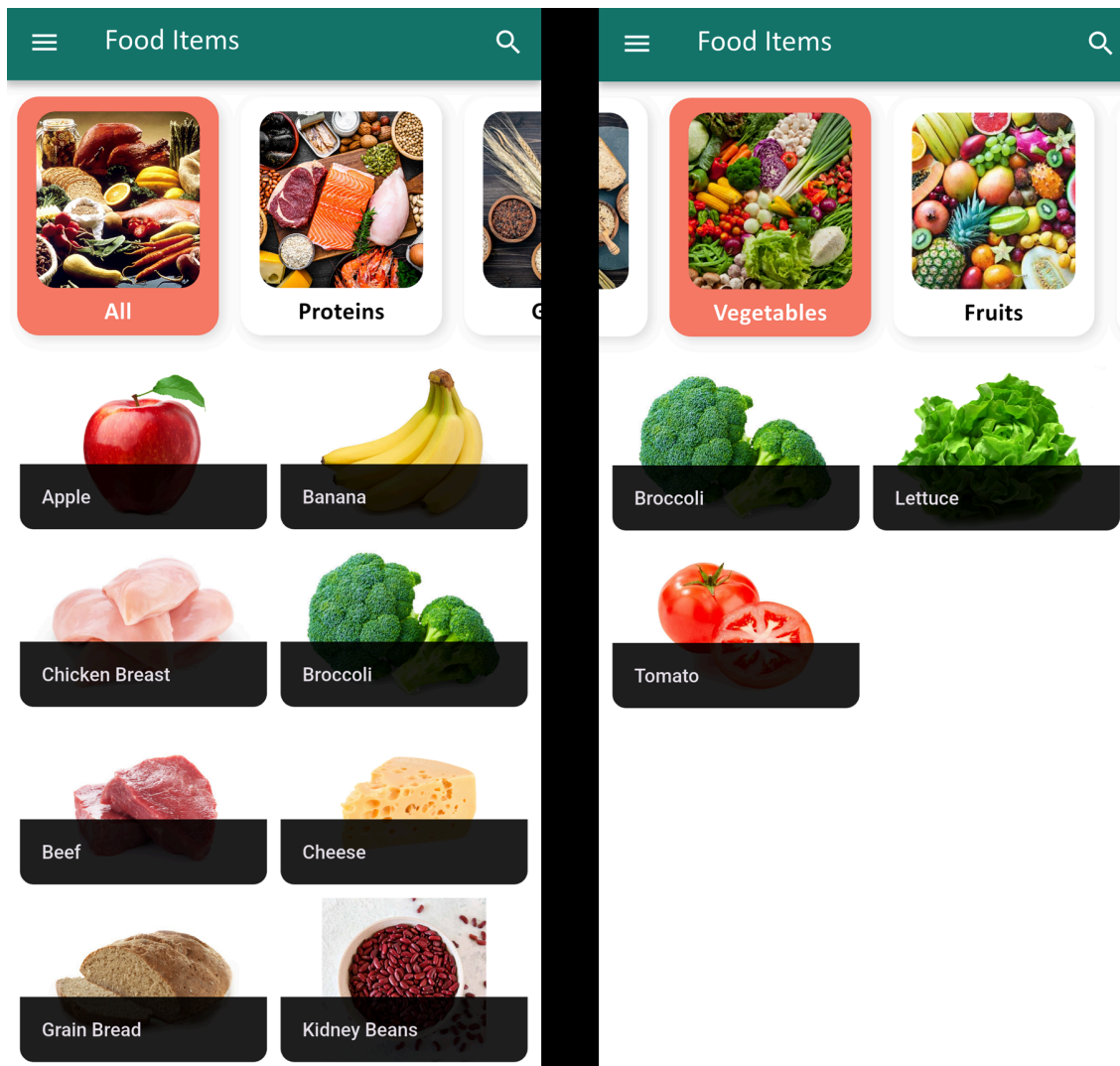
- This feature uses the following libraries and plugins:
 - **http:** Used for making HTTP requests, specifically in **UserService** for handling file uploads during user profile updates.
 - **pocketbase:** This is the core library for interacting with the PocketBase backend. It handles authentication, data storage, and retrieval.
 - **provider:** Used for state management, providing a way to share and update the user's authentication state across the application.
 - **flutter/material.dart:** The standard Flutter material design library, providing UI components.
- This feature uses the **provider** for state management.
 - The feature uses the **provider** package for shared state management.
 - The **UserManager** class extends **ChangeNotifier**, which allows it to notify listeners (widgets) when its state changes.
 - The **UserManager** holds the **_loggedInUser** state, which represents the currently authenticated user.
 - Widgets that need access to the user's authentication state use **context.read/watch<UserManager>()** or to access and listen to changes in the **UserManager**.
 - The **onAuthChange** callback in the **UserService** updates the **_loggedInUser** in the **UserManager** whenever the authentication state changes in PocketBase.
 - The **UserManager** is responsible for handling user authentication related actions, and then for notifying the UI of state changes.
- Data reading and storage of this feature:
 - **PocketBase (Remote):**
 - **Data Table Structure:** The code interacts with a "users" collection in PocketBase. The user object contains fields such as username, email, name, birthday, gender, role, password, and profileImage.
 - **REST API:** The **pocketbase** library abstracts the REST API calls. It makes requests to the PocketBase API.
 - **pb.collection('users').authWithPassword(username, password):** Sends a POST request to authenticate user.
 - **pb.collection('users').create(body: {...}):** Sends a POST request to create a new user.
 - **pb.collection('users').getOne(id):** Sends a GET request to retrieve a user by ID.
 - **pb.files.getUrl(userModel, imageName).toString():** Generates a URL to retrieve a user's profile image.

- **Auth Store (Local):**

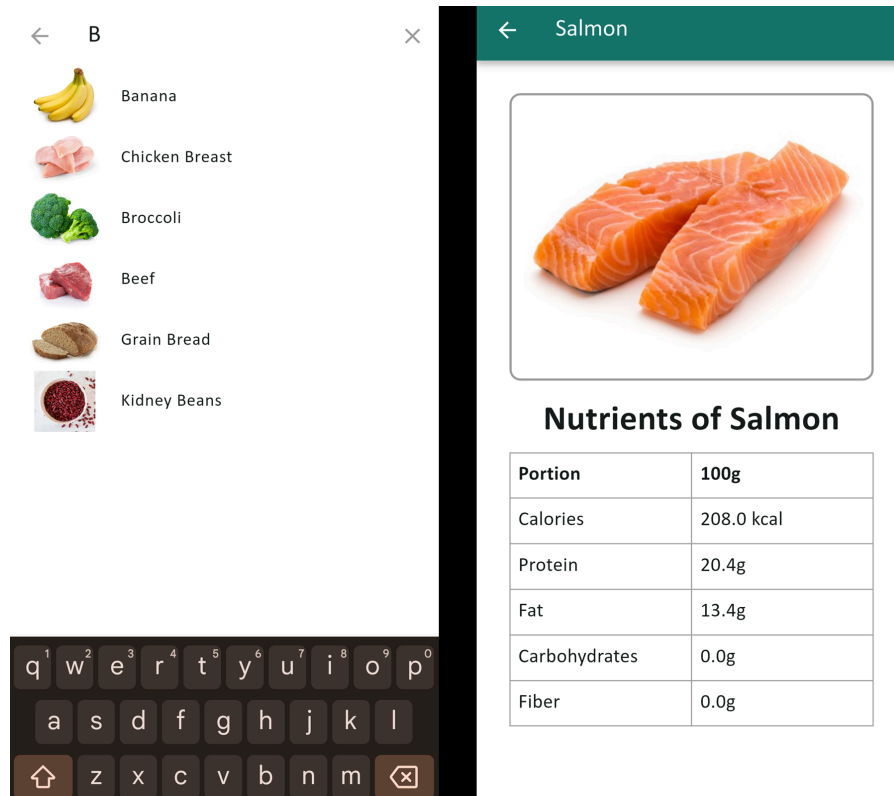
- PocketBase's **authStore** stores the user's authentication token locally.
- The **UserService** uses **pb.authStore.record** to retrieve the currently authenticated user's data.

2. Foods:

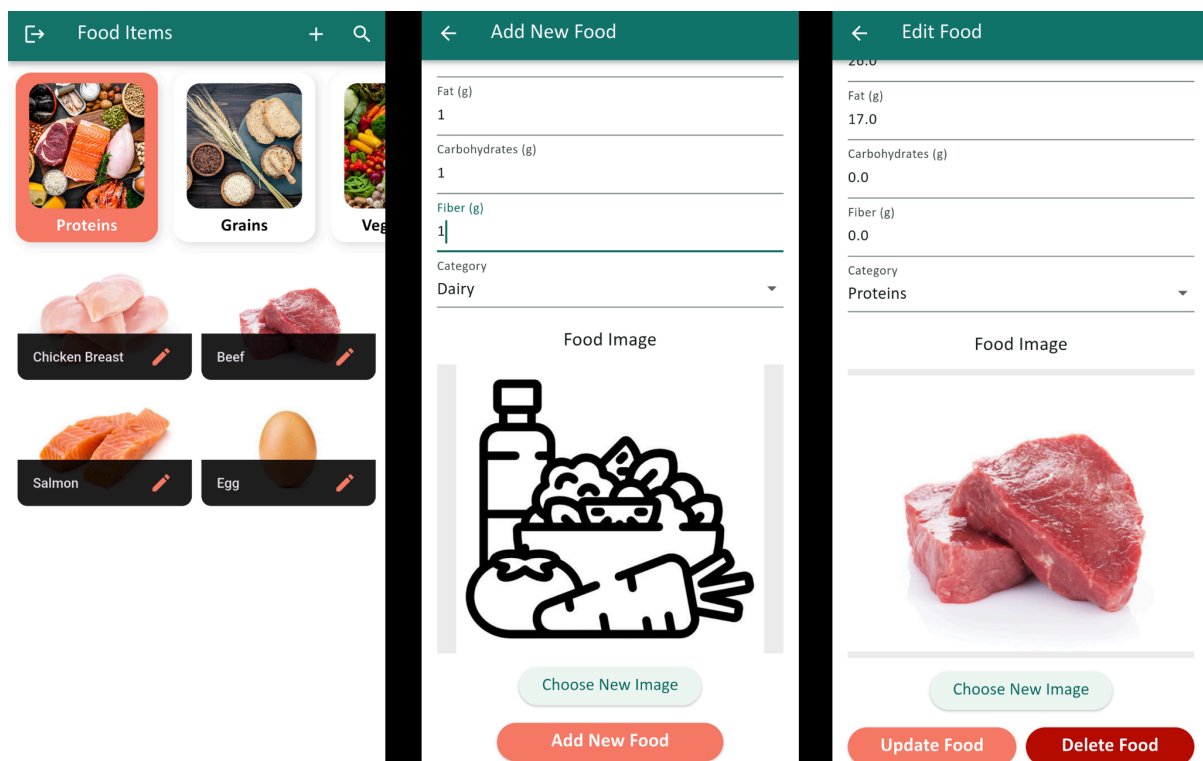
- **Description:** This feature displays detailed nutrition stats for various foods, allowing users to create their own meal recipes throughout the week. If logged in with an admin account, users can add, edit and delete food items.
- **Screenshots:**



Food Overview screen with category filters



Search delegate and Food Detail screen



Food Update screens for Admin

- **Implementation details:**

- Widgets used for this feature:

- **FoodsOverviewScreen** (Main screen)
 - Scaffold, AppBar, PopScope (Main screen layout and navigation control)
 - Drawer (Navigation menu for non-admin users)
 - CircularProgressIndicator (Loading state)
 - Column (Screen layout structure)
 - FoodTypeFilterMenu (Category selection)
 - Expanded, FoodsGrid (Displays food items)
- **FoodTypeFilterMenu** (Category selection menu)
 - SizedBox, ListView.builder (Horizontal scrollable categories)
 - GestureDetector (Handles category selection)
 - Container (Holds category name and image)
 - ClipRRect, Image.asset (Category image)
 - Text (Category name)
- **FoodsGrid** (Food item grid)
 - GridView.builder (Displays food items in grid format)
 - SliverGridDelegateWithFixedCrossAxisCount (Controls grid layout)
 - FoodGridTile (Individual food item display)
- **FoodGridTile** (Individual food item tile)
 - ClipRRect, GridTile (Rounded corners and structured tile layout)
 - GestureDetector (Opens food detail screen on tap)
 - Image.network, Image.asset (Food image display with fallback)
 - GridTileBar (Footer with food name and edit button for admin)
 - IconButton (Edit button for admin)
- **FoodSearchDelegate** (Search functionality)
 - SearchDelegate (Handles search UI)
 - IconButton (Clear search query, back navigation)
 - ListView.builder (Displays search results)
 - ListTile (Individual search result item)
 - Image.network, Image.asset (Food image display in search results)

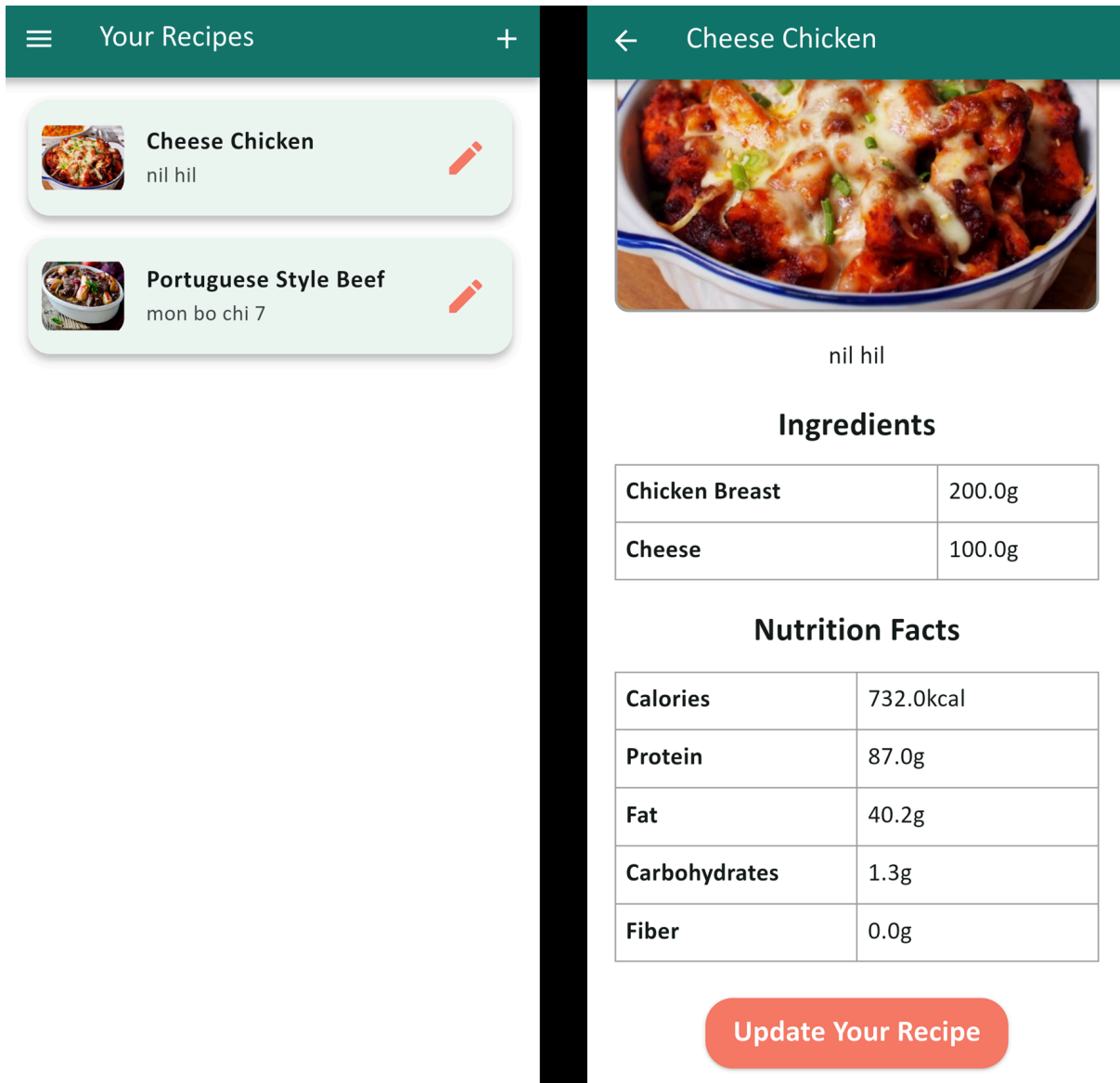
- **FoodDetailScreen** (Food detail view)
 - Scaffold, AppBar (Main screen structure)
 - SingleChildScrollView, Column (Scrollable layout structure)
 - Container, ClipRRect, Image.network, Image.asset (Food image display with fallback)
 - Text (Food name and title)
 - Table, TableRow, Padding, Text (Nutrient table)
 - Visibility, ElevatedButton (Edit button for admins)
- **FoodEditScreen** (Food edit/add screen)
 - Scaffold, AppBar (Main screen structure)
 - SingleChildScrollView, Padding, Form (Form layout)
 - TextFormField (Name, Calories, Protein, Fat, Carbohydrates, Fiber)
 - DropdownButtonFormField (Food category selection)
 - Column, Container, Image.file, Image.network, Image.asset (Food image display and picker)
 - ElevatedButton (Pick image, Submit form, Update food, Delete food)
- This feature uses the following libraries:
 - **http:** Used for making HTTP multipart requests, especially for uploading images when adding or updating food items.
 - **pocketbase:** The primary library for interacting with the PocketBase backend. It handles data retrieval, storage, and file management.
 - **provider:** Used for state management, enabling the sharing of food data and user authentication status across the application.
 - **flutter/material.dart:** The standard Flutter material design library, providing UI components.
 - **image_picker:** Used for selecting images from the device's gallery or camera.
- This feature uses the **provider** for state management.
 - The **FoodsManager** class extends **ChangeNotifier**, which allows it to notify listeners (widgets) when its state changes.
 - The **FoodsManager** holds the **_items** state, which represents the list of food items.

- Widgets that need access to the food data will use **context.watch<FoodsManager>()** to listen to changes in the **FoodsManager**.
- The **UserManager** class also utilizes provider to manage the users authentication status.
- The **FoodsManager** is responsible for fetching, adding, updating, and deleting food items, and then for notifying the UI of state changes.
- Data reading and storage of this feature:
 - **PocketBase (Remote):**
 - **Data Table Structure:** The code interacts with the "foods" collection in PocketBase. The food object contains fields such as id, name, calories, protein, fat, carbohydrates, fiber, type, and featuredImage.
 - **REST API:**
 - The **pocketbase** library abstracts the REST API calls. It makes requests to the PocketBase API.
 - **pb.collection('foods').getFullList():** Sends a GET request to retrieve all food items.
 - **pb.collection('foods').create(body: {...}, files: [...]):** Sends a POST request to create a new food item, including file uploads.
 - **pb.collection('foods').update(id, body: {...}, files: [...]):** Sends a PATCH request to update a food item, including file uploads.
 - **pb.collection('foods').delete(id):** Sends a DELETE request to delete a food item.
 - **pb.files.getUrl(foodModel, imageName).toString():** Generates a URL to retrieve a food item's image.
 - **Local File System:** The **FoodEditScreen** reads image files from the device's file system using the **image_picker** plugin.

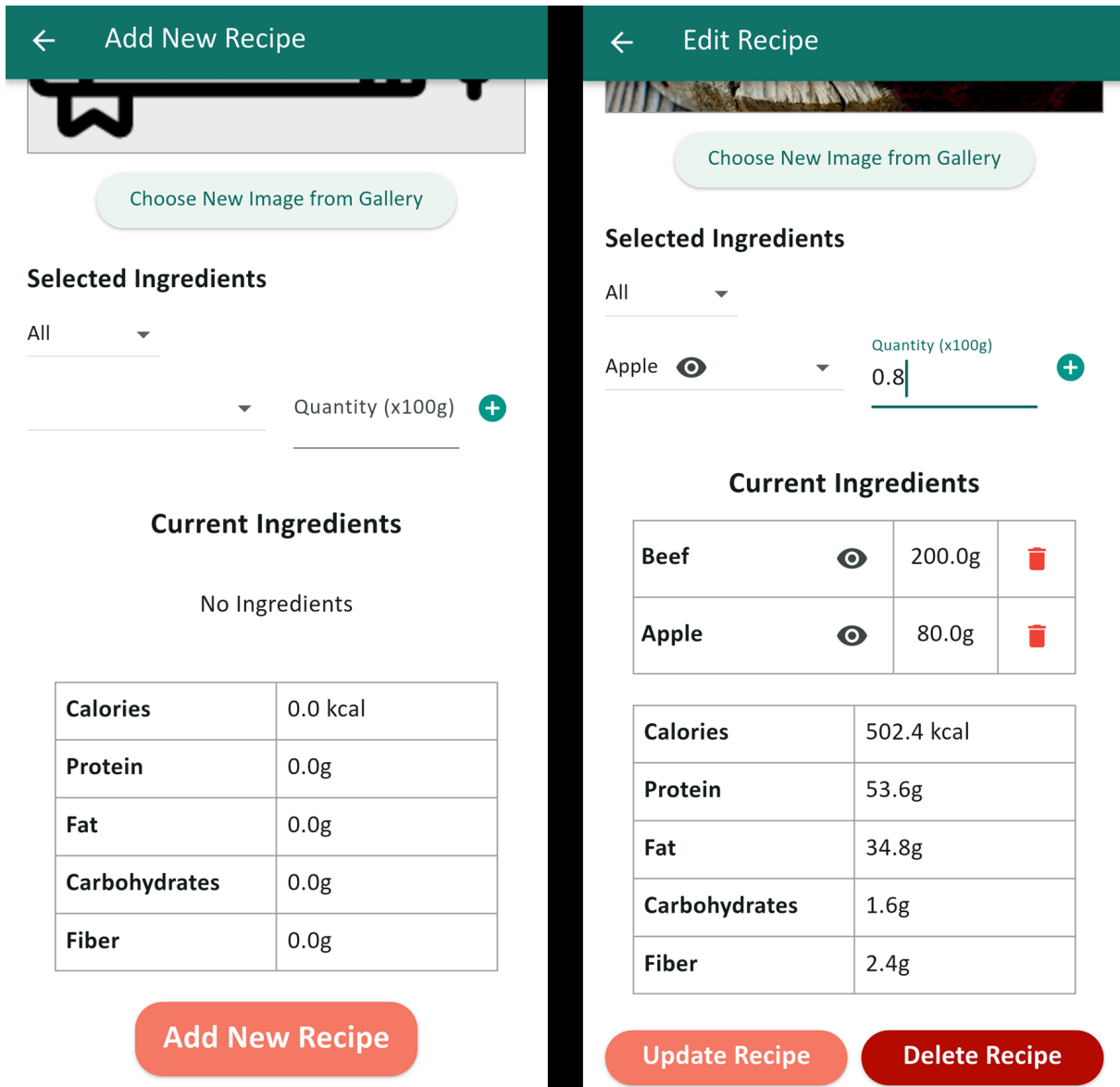
3. Recipes

- **Description:** This feature allows users to create and modify a weekly meal plan with three meals per day, displaying specific food items along with detailed nutrition information for each meal.

- Screenshots:



Recipes Overview & Recipe Detail screen



Recipe Edit screen

- Implementation details:

- Widgets used for this feature:

- RecipesOverviewScreen (Main screen)

- Scaffold, AppBar, Drawer (Main screen layout & navigation)
- FutureBuilder, CircularProgressIndicator (Handles async recipe fetching, loading state)
- ListView.builder (Displays user's recipes)
- RecipeListTile (Individual recipe display)

- **RecipeListTile** (Recipe item tile)
 - Dismissible (Swipe to delete)
 - Container, Icon (Delete background and trash icon)
 - Card, ListTile (Recipe display layout)
 - ClipRRect, Image.network, Image.asset (Recipe image with fallback)
 - Text (Recipe name, description)
 - IconButton (Edit button)
 - Navigator.pushNamed (Navigates to RecipeDetailScreen, RecipeEditScreen)
- **RecipeDetailScreen** (Recipe detail view)
 - Scaffold, AppBar (Main screen structure)
 - SingleChildScrollView, Column (Scrollable layout)
 - Container, ClipRRect, Image.network, Image.asset (Recipe image display with fallback)
 - Padding, Text (Recipe description)
 - Text (Ingredients title)
 - Container, Table, TableRow, Padding, Text (Ingredients table)
 - Text (Nutrition Facts title)
 - RecipeNutrientTable (Displays calculated nutrition values)
 - ElevatedButton (Update recipe button)
- **RecipeEditScreen** (Add/Edit Recipe Form)
 - Scaffold, AppBar (Main structure)
 - SingleChildScrollView, Column, Form (Scrollable form layout)
 - TextFormField (Recipe name & description)
 - Container, ClipRRect, Image (Recipe image)
 - ElevatedButton (Pick image)
 - DropdownButton, TextFormField, IconButton (Food selection & add ingredient)
 - Table, TableRow, IconButton (Ingredients list)
 - RecipeNutrientTable (Nutrition facts)
 - ElevatedButton (Add/Update/Delete recipe)
- **RecipeNutrientTable** (Nutrient table component)
 - Container, Table, TableRow, Padding, Text (Displays nutrient values)
 - TextFormField (Editable fields when isEditable is true)
- This feature uses the following libraries:
 - **http**: Used for making HTTP multipart requests, especially for uploading images when adding or updating food items.

- **pocketbase:** The primary library for interacting with the PocketBase backend. It handles data retrieval, storage, and file management.
- **provider:** Used for state management, enabling the sharing of food data and user authentication status across the application.
- **flutter/material.dart:** The standard Flutter material design library, providing UI components.
- **image_picker:** Used for selecting images from the device's gallery or camera.
- This feature uses the **provider** for state management.
 - The **RecipesManager** class extends **ChangeNotifier**, which allows it to notify listeners (widgets) when its state changes.
 - The **RecipesManager** holds the **_items** state, which represents the list of recipes.
 - The **FoodsManager** class also extends **ChangeNotifier** to manage the food items.
 - The **UserManager** class extends **ChangeNotifier** to manage the User authentication.
 - Widgets that need access to the recipe data use **context.watch<RecipesManager>()** to listen to changes in the **RecipesManager**.
 - The **RecipesManager** is responsible for fetching, adding, updating, and deleting recipes, and then for notifying the UI of state changes.
- Data reading and storage of this feature:
 - **PocketBase (Remote):**
 - **Data Table Structure:** The code interacts with the "recipes" and "foods" collections in PocketBase.
 - The "recipes" collection contains fields such as id, name, description, ingredients (which contains food id, and quantity), featuredImage, imageUrl, and userId.
 - The "foods" collection contains fields such as id, name, calories, protein, fat, carbohydrates, fiber, type, and featuredImage.
 - **REST API:**
 - The **pocketbase** library abstracts the REST API calls. It makes requests to the PocketBase API.
 - **pb.collection('recipes').getFullList():** Sends a GET request to retrieve all recipe items.
 - **pb.collection('recipes').create(body: {...}, files: [...]):** Sends a POST request to create a new recipe item, including file uploads.

- **pb.collection('recipes').update(id, body: {...}, files: [...]):** Sends a PATCH request to update a recipe item, including file uploads.
- **pb.collection('recipes').delete(id):** Sends a DELETE request to delete a recipe item.
- **pb.files.getUrl(recipeModel, imageName).toString():** generates a url to retrieve the image associated with the recipe.
- **pb.collection('foods').getFullList():** Sends a GET request to retrieve all food items.
- **Local File System:** The **RecipeEditScreen** reads image files from the device's file system using the **image_picker** plugin.

4. User Information

- **Description:** This feature allows users to view their profile, edit personal information, and change their account password.
- **Screenshots:**

The image displays three mobile application screens related to user management, each with a dark green header bar.

- User Profile Screen:** Features a title "User Information" and a list of input fields for "Username" (lambobby007), "Full Name" (Lâm Đẹp Trai), "Gender" (Male), "Date of Birth" (30/1/2000), and "Email Address" (lambobby007@gmail.com). At the bottom, there are two buttons: "Change Information" with a pencil icon and "Change Password" with a lock icon, both with right-pointing chevrons.
- Edit Profile Screen:** Features a title "Update Information", a circular profile picture of a group of people, and input fields for "Name" (Lâm Đẹp Trai), "Date of Birth" (30/01/2000), and "Gender" (Male, shown as a dropdown menu). A red "Save Changes" button is at the bottom.
- Change Password Screen:** Features a title "Change Password" and three input fields for "Old Password", "New Password", and "Retype New Password". A red "Save Changes" button is at the bottom.

User Information screens

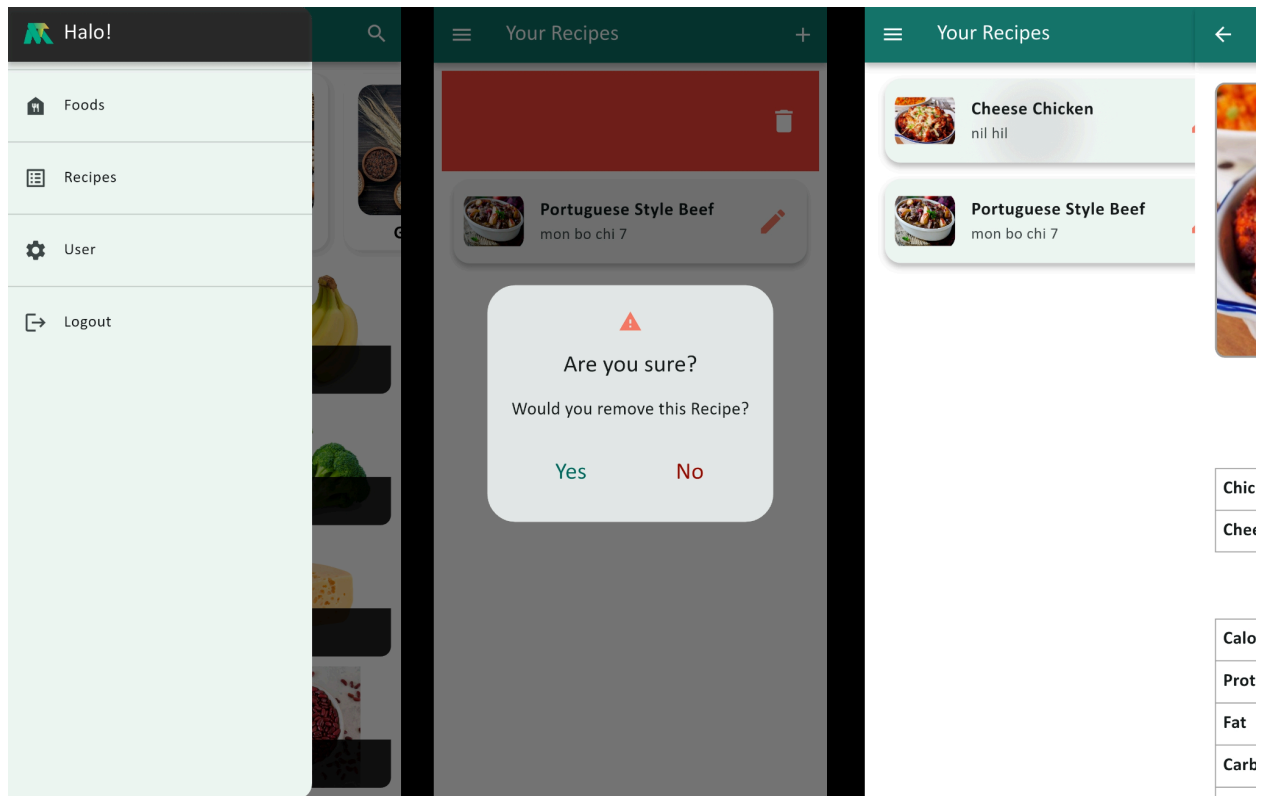
- **Implementation details:**

- Widgets used for this feature:
 - **UserProfileScreen** (User Profile & Settings)
 - Scaffold, AppBar (Main structure)
 - Drawer (AppDrawer) (Navigation menu)
 - Center, Text (No user logged in state)
 - SingleChildScrollView, Padding, Column (Profile layout)
 - CircleAvatar (Profile image)
 - Text (User information title)
 - SizedBox (Spacing between elements)
 - Container, Text (User details fields)
 - Column, Padding, ListTile (Settings section)
 - Icon, IconButton (Edit & Change Password buttons)
 - **UserUpdateInformationScreen** (Edit User Info)
 - Scaffold, AppBar (Main structure)
 - SingleChildScrollView, Padding, Column (Form layout)
 - Stack, CircleAvatar, IconButton (Profile image & edit button)
 - Text (Section title)
 - SizedBox (Spacing)
 - Form, Column, TextFormField (Input fields: Name, Birthday)
 - GestureDetector, AbsorbPointer (Date picker field)
 - DropdownButtonFormField (Gender selection)
 - ElevatedButton (Submit button)
 - **UserUpdatePasswordScreen** (Change Password Form)
 - Scaffold, AppBar (Main structure)
 - SingleChildScrollView, Padding, Form, Column (Form layout)
 - TextFormField (Old, New, Confirm Password)
 - SizedBox (Spacing)
 - ElevatedButton (Submit button)
- This feature uses the following libraries and plugins:
 - **http**: Used for making HTTP requests, specifically in **UserService** for handling file uploads during user profile updates.
 - **pocketbase**: This is the core library for interacting with the PocketBase backend. It handles authentication, data storage, and retrieval.
 - **provider**: Used for state management, providing a way to share and update the user's authentication state across the application.
 - **flutter/material.dart**: The standard Flutter material design library, providing UI components.

- **image_picker**: Used for selecting images from the device's gallery.
- This feature uses a the **provider** for state management.
 - The feature uses the **provider** package for shared state management.
 - The **UserManager** class extends **ChangeNotifier**, which allows it to notify listeners (widgets) when its state changes.
 - The **UserManager** holds the **_loggedInUser** state, which represents the currently authenticated user.
 - Widgets that need access to the user's authentication state use **context.read/watch<UserManager>()** or to access and listen to changes in the **UserManager**.
 - The **UserManager** is responsible for handling user authentication related actions, and then for notifying the UI of state changes
- Data reading and storage of this feature:
 - **PocketBase (Remote):**
 - **Data Table Structure:** The code interacts with the "users" collection in PocketBase. The user object contains fields such as username, email, name, birthday, gender, role, password, and profileImage.
 - **REST API:**
 - The **pocketbase** library abstracts the REST API calls. It makes requests to the PocketBase API.
 - **pb.collection('users').getOne(id)**: Sends a GET request to retrieve a user by ID.
 - **pb.collection('users').update(id, body: {...}, files: [...])**: Sends a PATCH request to update a user's data, including file uploads.
 - **pb.files.getUrl(userModel, imageName).toString()**: Generates a URL to retrieve a user's profile image.
 - **Auth Store (Local):**
 - PocketBase's **authStore** stores the user's authentication token locally.
 - The **UserService** uses **pb.authStore.record** to retrieve the currently authenticated user's data.
 - **Local File System:** The **UserUpdateInformationScreen** reads image files from the device's file system using the **image_picker** plugin.

5. Utils

- **AppDrawer:** Main navigation menu used across the app for quick access to different sections
 - **Foods:** Navigates to **FoodsOverviewScreen**
 - **Recipes:** Navigates to **RecipesOverviewScreen**
 - **User:** Navigates to **UserProfileScreen**
 - **Logout:** Closes the drawer, navigates to the home screen (/), and triggers **UserManager.logout()**
- **DialogUtils:** Utility for confirmation and error dialogs across the app
 - **showConfirmDialog** (Reusable confirmation prompt): Displays a warning icon, a message, and two action buttons Yes and No.
 - **showErrorDialog** (Reusable error notification): Displays an error icon, a message, and a single button to dismiss
 - **ActionButton** (Reusable button component): Custom-styled button with configurable text, color, and action
- **NavigationUtils:** A custom page transition animation for screen navigation that enhances the user experience when switching screens. It uses **Curves.easeInOut** for smooth motion, applied with **PageRouteBuilder**



AppDrawer, Dialog Utils and Navigation Utils